

Production Prefill/Decode disaggregation based on VLLM in Tencent



🔗 目录

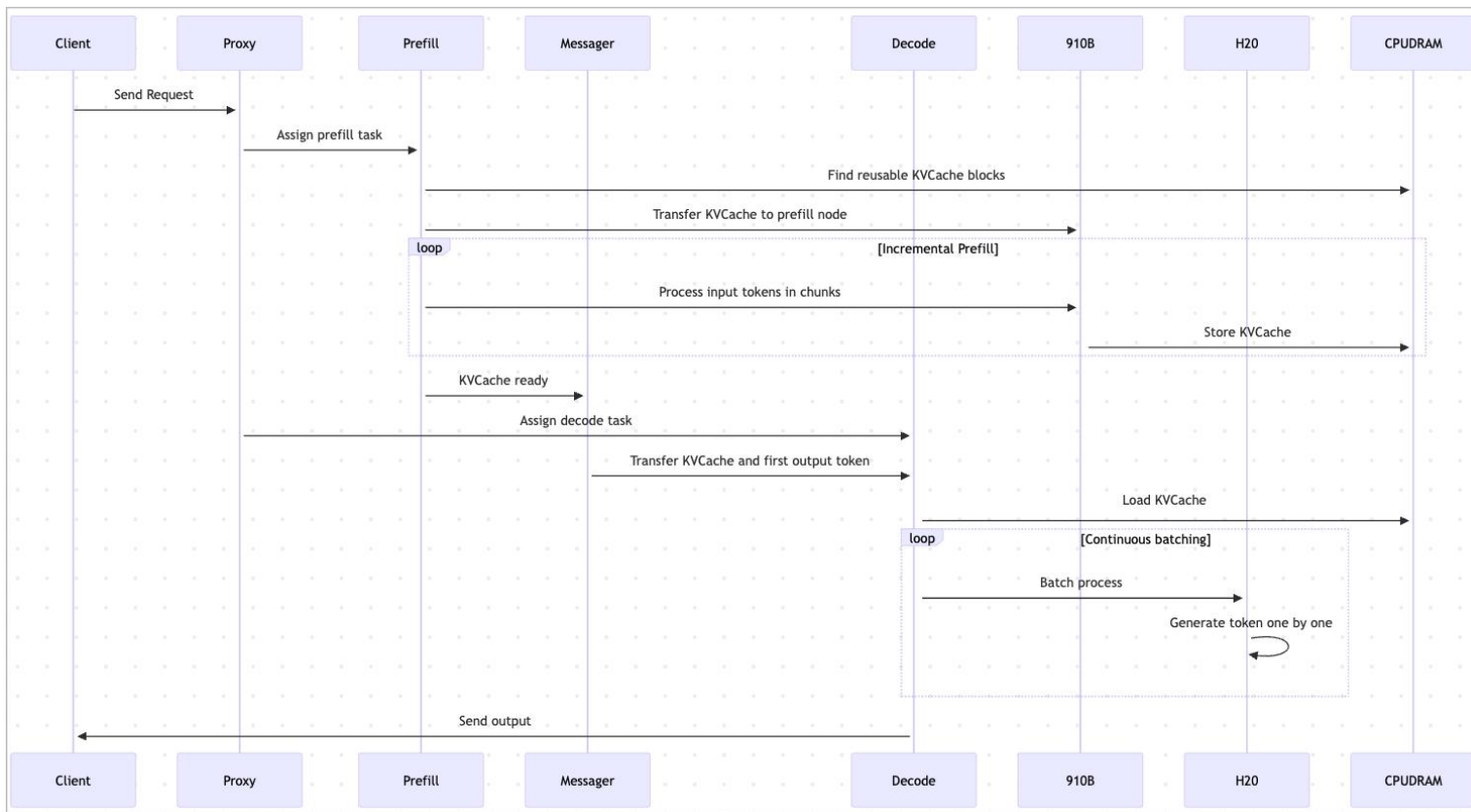
1. PD分离简介
2. 同构卡PD分离实践
3. 异构卡PD分离实践
4. 在VLLM上的其他工作

🔗 PD分离简介

- 动机
 - Prefill 阶段：计算密集型，并行处理全部输入，占用大量 GPU 算力但显存需求较低；batch size一般都在个位数，大batch size对性能提升无效
 - Decode 阶段：内存密集型，依赖高带宽显存频繁读取 KV 缓存，算力需求低但显存占用随上下文增长线性上升；通常通过增大batch size获得高吞吐
 - 混合部署性价比较低，难以保证Time To First Token(TTFT)和Time Per Output Token(TPOT)

	计算复杂度		并行粒度		访存		整体瓶颈
	token数	主要影响因素	提高并行度方式	并行方法	读/写	侧重点	
Prefill	所有输入token	输入长度	增加实例数	流水/分块并行	主要是写	带宽	计算资源
Decode	一次处理一个token	batch size	增加batch size	tensor并行	主要是读	容量/带宽	访存容量/带宽

PD分离实现--整体流程



🔄 PD分离实现—数据传输

- 用户请求信息
 - Proxy → Prefill
 - Proxy → Decode
 - HTTP
- Metadata
 - tensor.shape
 - tensor.dtype
 - token_id
 - TCP
- kv cache/hidden states
 - tensor data
 - TCP
 - NCCL
 - RDMA
 - Mooncake
 - LMCache

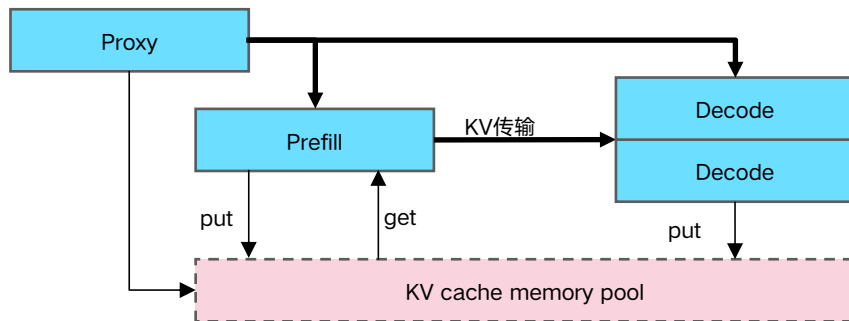
🕒 PD分离实现的一些权衡

- Proxy何时决定Prefill对应的Decode
 - Prefill和Decode同时确定
 - Prefill结束时确定Decode
 - 在Prefill执行过程中确定
- PD之间直连传输、通过 KV Cache Memory Pool
 - NCCL、TCP、RDMA等
 - LMCache、Mooncake等
- 按层发送/接收 KV Cache
 - `send_per_layer(kv_cache)`
 - `send_kv_caches_and_hidden_states(kv_caches, hidden_or_intermediate_states)`
- 首字来自Prefill还是Decode节点
 - Prefill: token_id, 传输的数据量小, ttft小, decode逻辑复杂
 - Decode: hidden_states, prefill无需加载logits相关的权重, ttft包括kv cache传输的时间, decode逻辑相对简单

🕒 PD分离实现的一些权衡

- Prefill和Decode节点动态扩缩容、相互转换，去中心化网络链接
 - 容错
 - 动态调整PD配比
- Prefix cache及Decode是否能回填KV Cache Memory Pool
 - 多轮对话场景
- 多级KV Cache Memory Pool
- KV Cache Buffer在Prefill还是Decode上
 - Decode节点后台进程负责从Buffer拷贝到kv cache
- 采用Push还是Pull方式
- 节点内同类型卡，节点间同类型，节点间不同类型卡，节点间不同厂家卡

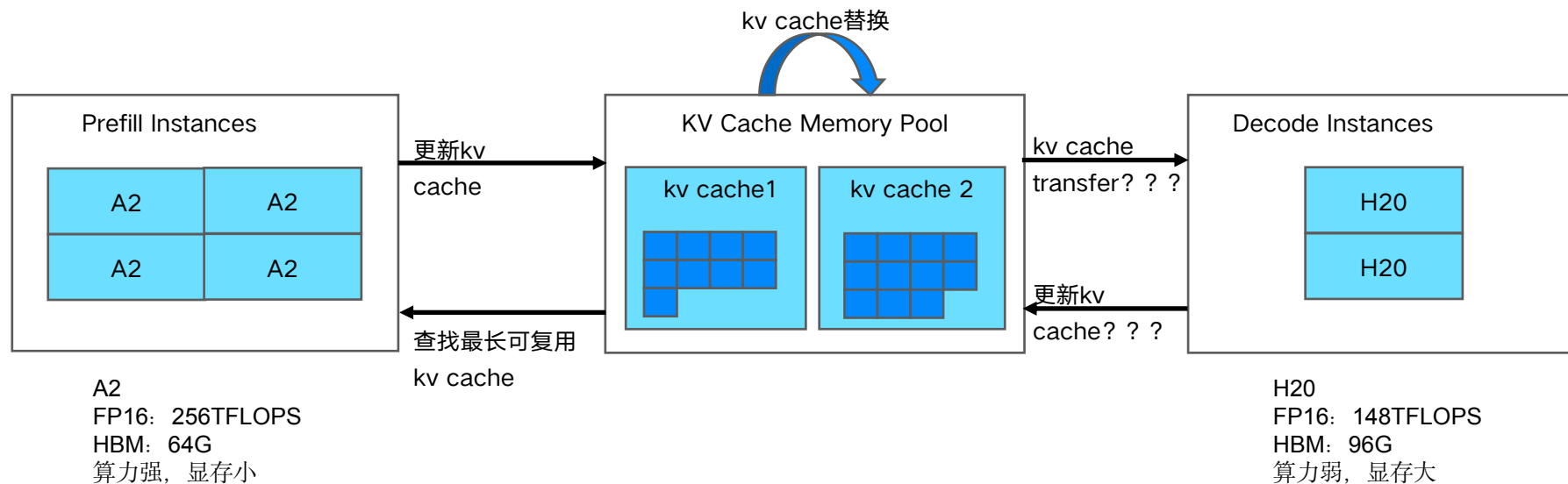
同构卡PD分离实践



- H20(5P8D, 共13*8=104卡)
- 支持满血版DeepSeek(单个实例占用2个节点)和量化版本(单个实例占用1个节点)
- mPnD
- 支持PP、MTP、按层发送/接收
- 对比PD混合模式, 在满足SLA (TTFT<1.5s, TPOT<50ms)前提下, 吞吐提升20+%; 在低并发部分场景中, 吞吐也有提升
- 在DeepSeek场景中, 为DP/大EP模式提供了功能基础

	并发	QPM			备注
		PD混合	PD分离	提升率	
输入集1	8	42	40	-3.6%	4P6D
	16	66	64	-3.5%	
	32	84	99	18.6%	
输入集2	8	16	16	3.9%	3P3D
	16	21	24	12.7%	
	32	24	26	10.2%	

异构卡PD分离实践



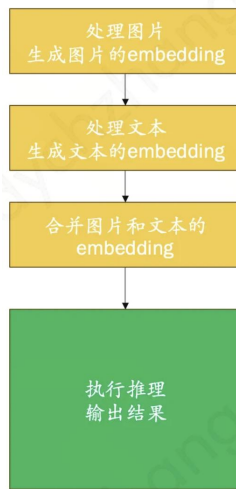
收益预估

- 发挥异构卡PD分离能力(H20:H20 --> A2:H20), 进一步提高性价比
- 整合通讯库, 统一不同介质间的通讯接口
- 增加KV Cache Memory Pool进一步加速Prefill阶段
- 相比全部H20场景, 异构卡PD分离场景中通讯开销增大很多, 分层发送/接收

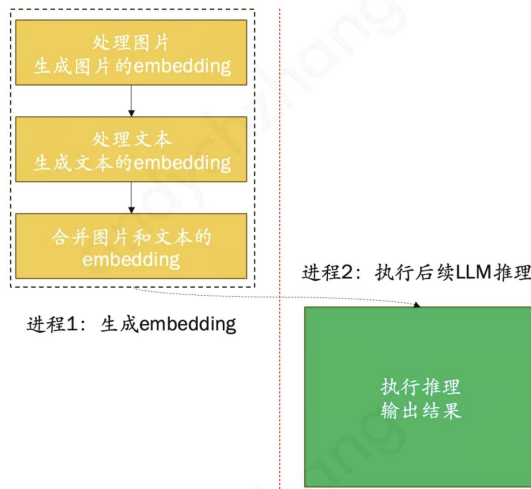
- 以input_len: output_len = 1024: 128为例, 达到最佳PD配比后折算总卡时:
 - A2+H20 对比H800 PD混布Baseline 收益提升50%
 - A2+H20 对比H800+H20 收益提升20%

在VLLM上的其他工作--CPU

- 内存资源紧张，计算资源相对宽松
- 模型拆分，多模态模型拆分成生成Embedding和推理结果输出两部分
 - 流水线并行，更充分利用计算资源
 - 一个模型变两个模型，启动更快
- BF16 KV cache
- OpenMP并发动态调整
- GPU上通常使用16位甚至更短的数据类型，但是大部分CPU不支持16位数据类型，在CPU上模拟16位数据类型效率非常低，使用FP32+向量化
- AVX256支持
- 已落地Intel/AMD多个型号CPU集群，单个业务落地数十万core



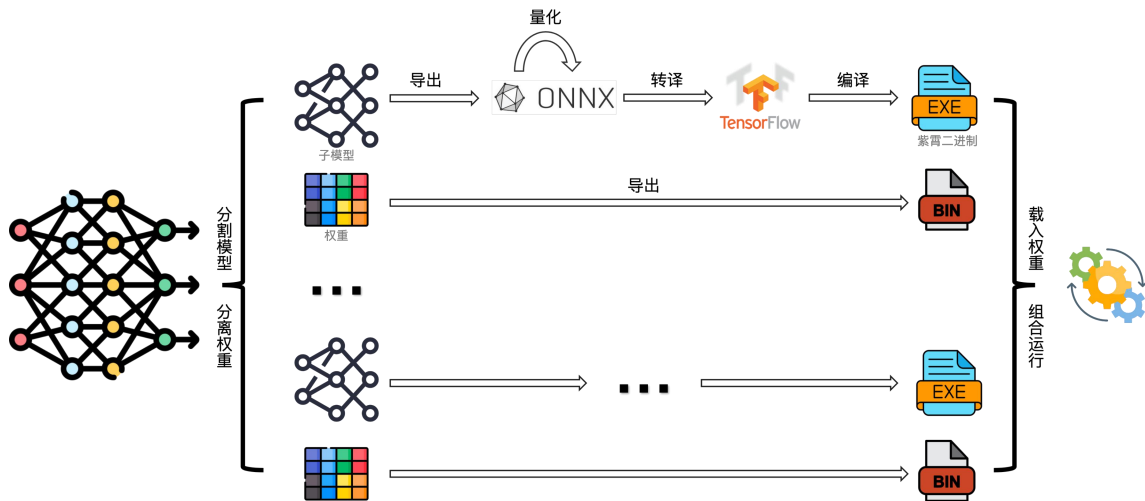
(a) 单进程串行执行



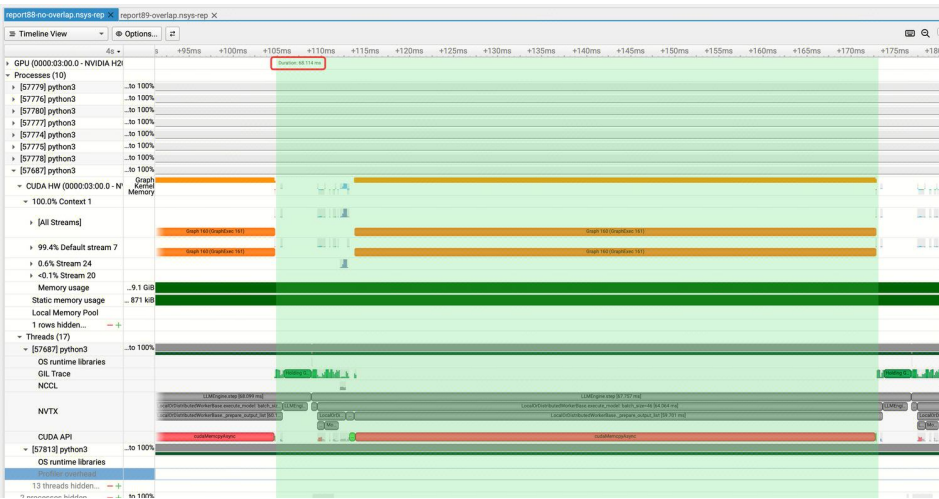
(b) 双进程流水线并行执行

在VLLM上的其他工作—某国产异构芯片

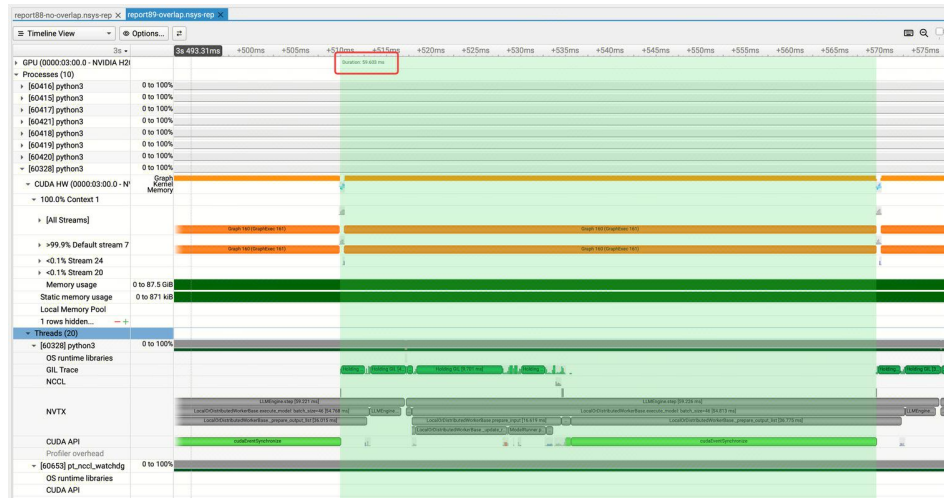
1. 基于VLLM，支持了某国产异构芯片V1和V2版本
2. V1提供了Embedding、MHA、MLP、LM等大粒度融合算子及add、multiply、Gemm等算子
3. V2提供了Gemm, Norm, Attention等细粒度算子
4. 完成了Pytorch模型到可执行代码的图编译器
 1. 大模型场景尽量复用融合算子，对于频繁修改的部分结构使用图编译快速生成算子
 2. 支持非大模型场景模型



在VLLM上的其他工作--GPU



68.114ms



59.603ms

CPU/GPU Overlap 10%~15%



非常感谢 ~

欢迎大家加入
andychzhang@tencent.com